Final Folder Structure

🧱 SmartEdu Project – Folder Structure Documentation

```
SMART-EDU-KNOWLETIVE-PROJECT/
│
├── .vscode/
├── client/
├── Diagrams-For-Visualizations/
├── docker/
├── docs/
├── Documents-Image-Files-For-Reference/
├── PostgreSQL-DB/
├── server/
```

📁 `.vscode/`

Stores **VS Code editor settings** for the project.

Used for workspace configs like extensions, formatting rules, and debugging setup.

📁 `client/` (Frontend)

Contains the **entire frontend application** built with Next.js/React.

Handles UI, dashboards, pages, and user interaction.

```
client/
├── public/
├── src/
```

📁 `client/public/`

Stores **static assets** that are directly accessible by the browser.

Used for icons, images, logos, and public files.

```
public/
├── icons/
├── images/
```

`icons/`

Contains UI icons used across the application.

Example: navigation icons, feature icons, etc.

`images/`

Stores images like banners, illustrations, and landing page visuals.

These files are used directly in UI components.

📁 `client/src/`

Main **frontend source code directory**.

All pages, components, features, and frontend logic live here.

```
src/
├── app/
├── components/
├── features/
├── hooks/
```

```
├── lib/
├── store/
├── styles/
└── types/
```

📁 `app/`

Handles **routing and page structure** using Next.js App Router.

Each folder represents a route in the application.

Example: `/login`, `/dashboard/student`.

📁 `components/`

Reusable **UI building blocks**.

Used across many pages like buttons, cards, navigation bars, etc.

📁 `features/`

Organizes the app **by business features**.

Each feature contains its own components, API calls, and logic.

Example: auth, tests, notes, AI features.

📁 `hooks/`

Contains **custom React hooks**.

These help reuse logic like authentication state or fetching user data.

📁 `lib/`

Utility tools for the frontend.

Usually contains API helpers, utility functions, and external integrations.

📁 `store/`

Stores **global state management logic**.

Used to manage user sessions, app state, and shared data across pages.

📁 `styles/`

Contains **global CSS and styling configuration**.

Used for layout styles, theme configuration, and UI consistency.

📁 `types/`

Defines **TypeScript types and interfaces**.

Ensures consistent data structures between components and API responses.

📁 `Diagrams-For-Visualizations/`

Contains architecture diagrams and system design visuals.

Used to explain workflows, database structure, and system flow.

📁 `docker/`

Stores **Docker configuration files**.

Used for containerizing the application and managing deployment environments.

📁 `docs/`

Contains **project documentation**.

Used for architecture explanations, setup guides, and developer notes.

📁 `Documents-Images-Files-For-Reference/`

Stores research materials and external references.

Helpful for project planning and documentation purposes.

📁 `PostgreSQL-DB/`

Contains **database schema and SQL scripts**.

Used for creating and maintaining the project database.

```
PostgreSQL-DB/
└── SmartEdu-DB.sql
```

`SmartEdu-DB.sql`

Main SQL file that defines the database structure.

Includes tables, relationships, and constraints for the SmartEdu system.

📁 `server/` (Backend)

Contains the **backend API and business logic**.

Handles authentication, database operations, AI processing, and system services.

```
server/
├── prisma_or_migrations/
├── src/
└── package.json
```

📁 `prisma_or_migrations/`

Stores **database migration files**.

Used to track database schema changes over time.

📁 `src/`

Main backend source code directory.

Contains configuration, modules, middleware, and utilities.

```
src/
├── config/
├── middleware/
├── modules/
├── types/
├── utils/
└── app.ts
```

📁 `config/`

Stores **backend configuration settings**.

Used to manage environment variables, database connection, and cloud setup.

```
config/
├── cloud.ts
├── db.ts
└── env.ts
```

`cloud.ts`

Handles cloud service configuration.

Used for integrations like storage services or external APIs.

`db.ts`

Configures the database connection.

Establishes communication between the backend and PostgreSQL.

`env.ts`

Loads environment variables.

Keeps secrets like API keys and database URLs secure.

📁 `middleware/`

Contains backend **request processing layers**.

Used for authentication, security checks, and error handling.

```
middleware/
├── auth.middleware.ts
├── error.middleware.ts
├── rateLimit.middleware.ts
└── role.middleware.ts
```

`auth.middleware.ts`

Verifies if a user is authenticated.

Checks JWT tokens before allowing access to protected routes.

`error.middleware.ts`

Handles application errors centrally.

Ensures consistent error responses across the API.

`rateLimit.middleware.ts`

Limits how often users can send requests.

Helps protect the server from abuse and brute-force attacks.

`role.middleware.ts`

Controls **role-based access**.

Ensures only authorized roles can access certain endpoints.

📁 `modules/`

Core backend feature modules.

Each module manages a specific system feature.

```
modules/
├── ai/
├── auth/
├── courses/
├── notes/
├── payments/
├── sessions/
├── tests/
└── users/
```

`ai/`

Handles AI-related functionality.

Includes logic for generating insights, analysis, or recommendations.

`auth/`

Manages authentication processes.

Includes login, registration, token creation, and password security.

`courses/`

Handles course-related operations.

Includes course creation, updates, and student enrollment in courses.

`notes/`

Manages educational notes and study materials.

Allows teachers to create and students to access notes.

`payments/`

Handles payment processing.

Manages subscription plans, transactions, and payment status.

`sessions/`

Manages learning sessions or classes.

Handles scheduling, session tracking, and participation data.

`tests/`

Handles assessments and test management.

Includes creating tests, storing results, and evaluating performance.

`users/`

Manages user accounts and profiles.

Handles user data like students, teachers, tutors, and admins.

📁 `types/`

Stores backend TypeScript type definitions.

Ensures consistent data structures across services and modules.

📁 `utils/`

Contains helper functions used across the backend.

Includes reusable logic like formatting, logging, and small utilities.

📄 `app.ts`

Main backend application entry point.

Initializes middleware, routes, and starts the API server.

📄 `package.json`

Manages project dependencies and scripts.

Used for installing packages and running backend commands.

🧠 Simple Big Picture

Frontend (`client/`) → UI and user interaction.

Backend (`server/`) → logic, database communication, and APIs.

Database (`PostgreSQL-DB/`) → stores all system data.